

Publicación de notas:

24 de junio de 2024

Revisión:

26 de junio de 2024 de 10:30-12:30

FIRMA

Apellidos: _____

Nombre: _____

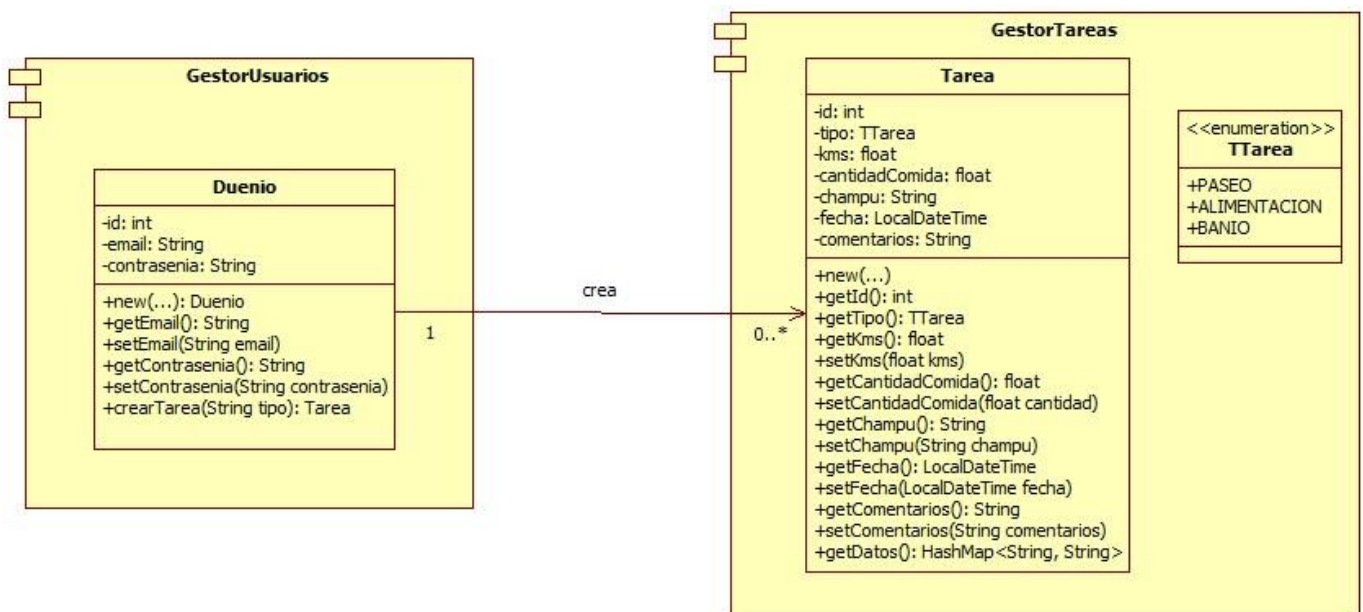
DNI: _____

PROBLEMA 2 (5 puntos)

Apartado 2.1 (4 puntos)

Diana es una estudiante de la ETSISI que está a punto de terminar la carrera y ha decidido crear una innovadora plataforma llamada Cuidando a Pancho, cuyo objetivo es facilitar que personas amantes de los perros puedan ofrecer sus servicios a personas que tienen un perro (en adelante, dueños), pero que no disponen de suficiente tiempo para atenderlo completamente.

Antes de realizar la aplicación completa, Diana ha querido abordar una parte crítica de la misma que está relacionada con la creación de tareas por parte de los dueños y el acceso de estos a la información de las tareas. A continuación, puedes ver un fragmento del diseño realizado para abordar esta parte de la aplicación. Como puedes observar, Diana tiene algo oxidados sus conocimientos sobre principios de diseño y patrones de diseño de software y el diseño actual del sistema presenta ciertos problemas. Entre otras cosas se puede observar que hay un alto acoplamiento entre componentes y que el sistema es poco extensible. **Mejora este diseño usando los principios de diseño que consideres y utilizando el patrón Factoría para el proceso de creación de tareas por parte de los dueños. Indica también los principios de diseño empleados en el recuadro proporcionado a tal efecto.**



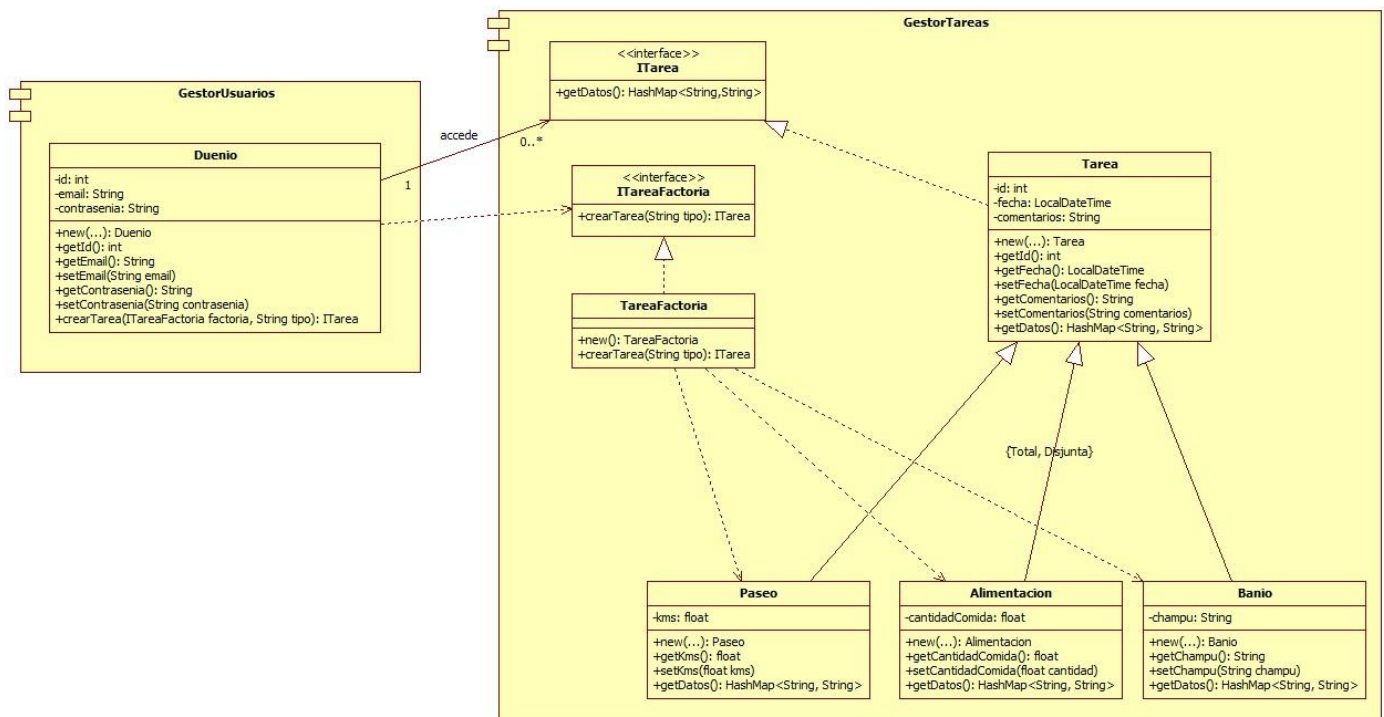
La solución diseñada debe cumplir lo siguiente:

- Respecto a la **creación de Tareas**, se debe respetar la definición actual de la clase Duenio, salvo en dos aspectos: el método crearTarea puede cambiar ya que sus parámetros de entrada/salida pueden modificarse; las relaciones con los elementos del componente GestorTareas también pueden modificarse. Además, tal y como se ha indicado antes se debe emplear el patrón Factoría para facilitar la creación de Tareas y se debe tener en cuenta que es la propia factoría la encargada de solicitar al usuario los datos específicos de cada tipo de Tarea (Duenio solo le facilitará a la factoría el tipo de Tarea a crear).
- Respecto al **acceso a los datos de las Tareas**, los dueños solo deben tener acceso de lectura a la información de las tareas a través del método getDatos(): HashMap<String, String>. Ten en cuenta que por el momento hay un conjunto limitado de Tareas que los dueños pueden crear (concretamente, paseos, alimentación, y baños), pero en el futuro se pretenden ofrecer más tareas como, por ejemplo, la administración de medicamentos.

SOLUCIÓN

Principios de diseño aplicados:

- Principio de responsabilidad única y cohesión
- Abstracción por especialización
- Principio Abierto/Cerrado
- Principio de Sustitución de Liskov
- Principio de Inversión de Dependencias



Apartado 2.2 (1 punto)

Diana ha comenzado el desarrollo y ha realizado la función registrarUsuario, cuyo código puedes ver a continuación. **Construye el grafo de flujo asociado a esta función aplicando la prueba de la ruta base e indica la complejidad ciclomática.**

```
public void registrarUsuario() {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Introduce tu correo electrónico: ");
    String correo = scanner.nextLine();
    int intentos = 3;
    String contrasenia = null;
    boolean registrado = false;
    while (intentos > 0 && !registrado) {
        System.out.print("Introduce una contraseña (mínimo 6 caracteres): ");
        String contraseniaIngresada = scanner.nextLine();

        if (contraseniaIngresada.length() >= 6) {
            contrasenia = contraseniaIngresada;
            registrado = true;
        } else {
            System.out.println("La contraseña es demasiado corta.");
        }
        intentos--;
    }

    if (registrado) {
        System.out.println("Usuario registrado correctamente.");
        // Sentencias para almacenar el usuario (supón que no hay sentencias de control de flujo)
        ...
    } else {
        System.out.println("Has superado el número máximo de intentos. Registro cancelado.");
    }
}
```

SOLUCIÓN

La complejidad ciclomática es **5** (número de regiones, o número de nodos predicado +1, o aristas – nodos + 2)

